Collector Pipeline

Guide	Collector Pipeline
Team Owner	WHAT
Status	DEPRECATED
Last Modified	09 Sep 2019

- Theory
- How To
  - Enable / Disable
- Diagnosis and Mitigation

Introduction

This article has been deprecated. Please see: Dynamic App Collector Pipeline Troubleshooting Guide

I started this playbook to get the ball rolling on writing down how-to knowledge for support about collector pipeline. Any of the details here could change before we ship CPL for the first time, so if you happen to have read some of this before launch, be sure to re-read after launch to verify how things work.

Theory

How To

Enable / Disable

CPL is enabled / disabled per data type. This function is called by collection processes to determine if they should send data via data pull or over the CPL (G3code/backend/silo_common/misc.py):

```
@TimedMemoize(ttl=300)
def is_cpl_enabled(dbc, datatype):
    """
    Determines if collector pipeline is enabled for this data type.

    This leverages the datatypes defined in COLLECTION_TYPES in global_defintions.
    Subtracts 1 given that the datatypes start at 1 and not 0.
    """
    cpl_dtype_mask = int(dbc.autofetch_value("""
        SELECT enable_collector_pipeline
        FROM master.system_settings_core
    """))

    return bool(cpl_dtype_mask & (1 << (datatype - 1)))
```

From G3code/backend/silo_common/global_definitions.py, we have:

```
COLLECTION_TYPE_DEV_AVAIL = 1       #data_avail
COLLECTION_TYPE_CV_STATS = 2        #data_cv
COLLECTION_TYPE_DNS = 3            #data_dns
COLLECTION_TYPE_EMAIL_ROUNDTRIP = 4  #data_email
COLLECTION_TYPE_FTP = 5            #? data_ftp
COLLECTION_TYPE_INTERFACE = 6       #data_if
COLLECTION_TYPE_PORT_AVAIL = 7      #data_ports
COLLECTION_TYPE_PROCESS_CHECK = 8   #data_procs
COLLECTION_TYPE_SERVICE_CHECK = 9   #data_services
COLLECTION_TYPE_FS_STATS = 10       #data_storage, device_hardware
COLLECTION_TYPE_SVC = 11           #? data_svc
COLLECTION_TYPE_TV_STATS = 12       #data_tv
COLLECTION_TYPE_DYN_APP_CONFIG = 13  #dynamic_app_data_%s
COLLECTION_TYPE_DYN_APP_PERF = 14   #dynamic_app_data_%s
COLLECTION_TYPE_DYN_APP_JOURNAL = 15 #dynamic_app_data_%s
COLLECTION_TYPE_CERTIFICATES = 16   #?  populated by discover_detail_storage
COLLECTION_TYPE_FS_INVENTORY = 17   #device_hardware
```

```
COLLECTION_TYPE_TOPOLOGY = 18       #device_interfaces, device_interfaces_map
COLLECTION_TYPE_IP_ADDR = 19        #? device_ip_addr - discovery?
COLLECTION_TYPE_PORTS = 20          #? device_ports - discovery?
COLLECTION_TYPE_SERVICE = 21        #device_services
COLLECTION_TYPE_PROCESS = 22        #device_processes
COLLECTION_TYPE_SNMP_DETAIL = 23     #device_snmp_data
COLLECTION_TYPE_CV_CONTENT = 24      #content_sample
COLLECTION_TYPE_TV_CONTENT = 25      #trans_sample
COLLECTION_TYPE_DEV_AVAIL_CRITICAL = 26 #data_avail
COLLECTION_TYPE_L3_TOPOLOGY = 27     #device_l3_map
COLLECTION_TYPE_ITSM_AGGREGATION = 28 #itsm
COLLECTION_TYPE_CBQOS_INVENTORY = 29 #collect_cbqos_inventory
COLLECTION_TYPE_CBQOS_DISCOVERY = 30 #discover_cbqos
COLLECTION_TYPE_CBQOS_PERFORMANCE = 31 #collect_cbqos
COLLECTION_TYPE_SILO_METRICS = 32    #silo_metrics
```

As master.system_settings_core is being treated as a bitmask, enabling individual collection types is a matter of turning on and off bit position (COLLECTION_TYPE - 1).

Using mysql bitwise operators, we can set bit n with:

```
update master.system_settings_core set enable_collector_pipeline = enable_collector_pipeline | (
```

and we can reset bit n with

```
update master.system_settings_core set enable_collector_pipeline = enable_collector_pipeline & ~
```

For data_avail (type1) n=1 - 1=0, and for data_if, n= 6 - 1 = 5.

Diagnosis and Mitigation